Object Oriented Programming(C++)
(CSE 1211)

Constructor & Destructor

Md. Mahfuzur Rahman
Lecturer,
Prime University

Constructor

- What is a constructor ?
- Constructor is a special type of function that is used to initialize the object.

- What are the properties of constructor ?

1. Constructor is a special type of function.
2. Constructor has the same name as that of the class it belongs.
3. It has no return type not even void.
4. It is called automatically.

Constructor

```cpp
#include<iostream>
using namespace std;

class Student
    {
        public:
            int id;
            double cgpa;

            void display()
            {
                cout << id << " " << cgpa << endl;
            }

            Student(int x, double y)
            {
                id = x;
                cgpa = y;
            }
    };

int main()
{
    Student alif(101,3.80);
    alif.display();

    Student esha(102,3.90);
    esha.display();

    return 0;
}
```

Constructor

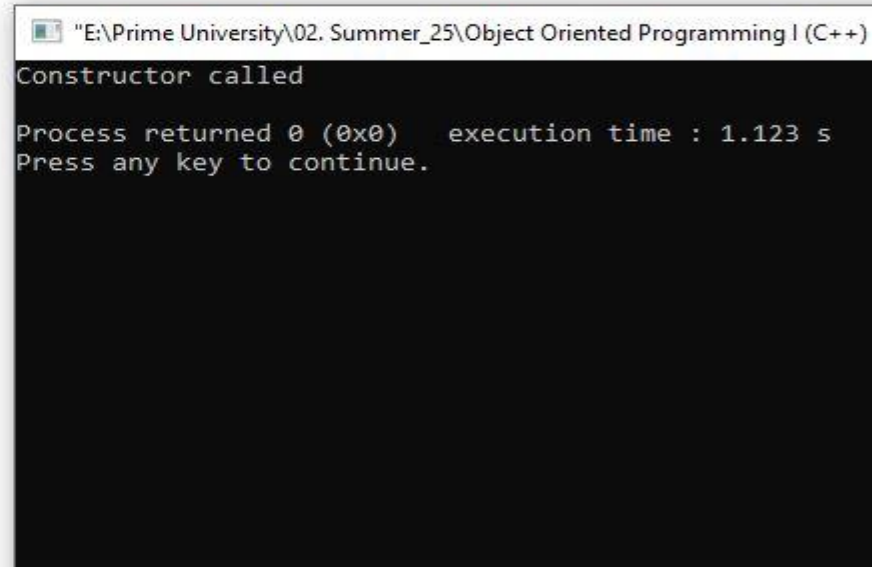
- A class **constructor** is a special member function of a class that is executed whenever we create new objects of that class.
- A constructor will have exact same name as the class and it does not have any return type at all, not even void. Constructors can be very useful for setting initial values for certain member variables.

```cpp
#include <iostream>
using namespace std;

class A {
public:

    A() {
        cout << "Constructor called" << endl;
    }
};

int main() {
    A obj1;
    return 0;
}
```



- In the above program, we have defined a constructor for **class A**. In the main function, when we create an object of that class, this constructor is called, which prints " Constructor called".

Types of Constructor

Constructors can be classified based on the situations they are being used in. There are 4 types of constructors in C++:

- **Default Constructor**
- **Parameterized Constructor**
- **Copy Constructor**

Default Constructor

A **default constructor** is automatically generated by the compiler if the programmer does not define one. This constructor doesn't take any argument as it is parameter less and initializes object members using default values. It is also called a zero-argument constructor. However, the default constructor is not generated by the compiler if the programmer has explicitly defined a constructor.

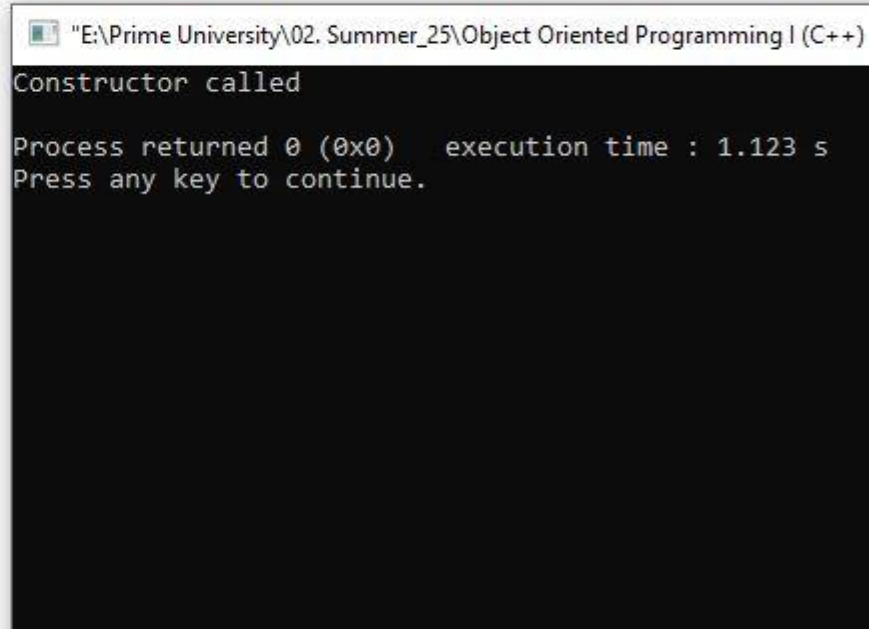
A **default constructor** is a constructor that does not take any parameters or has default values for all parameters.

```cpp
#include <iostream>
using namespace std;

class A {
public:

    A() {
        cout << "Constructor called" << endl;
    }
};

int main() {
    A obj1;
    return 0;
}
```



Parameterized Constructor

- **Parameterized constructor** allow us to pass arguments to constructors. Typically, these arguments help initialize an object's members. To create a parameterized constructor, simply add parameters to it the way you would to any other function. When you define the constructor’s body, use the parameters to initialize the object's members.
- **C++ parameterized constructor** is a constructor, which accepts one or more arguments or parameters, to initialize an object with specific values, when created. Unlike the default constructor, which takes no arguments, a parameterized constructor takes one or more arguments.

Parameterized Constructor

```cpp
#include<iostream>
using namespace std;

class Student
    {
        public:
            int id;
            double cgpa;

            void display()
            {
                cout << id << " " << cgpa << endl;
            }

            Student(int x, double y)
            {
                id = x;
                cgpa = y;
            }
            Student()
            {
                cout << "This is Default Constructor";
            }
    };
```

```cpp
int main()
{
    Student alif(101,3.80);
    alif.display();

    Student esha(102,3.90);
    esha.display();

    Student obj;

    return 0;
}
```

Destructor

- Destructors in C++ are members functions in a class that delete an object. They are called when the class object goes out of scope such as when the function ends, the program ends, a delete variable is called etc.
- Destructors are different from normal member functions as they don’t take any argument and don’t return anything. Also, destructors have the same name as their class and their name is preceded by a tilde(~).

- A destructor is a special member function that is called automatically when an object goes out of scope or when we delete the object with the delete expression.
- In C++, a destructor has the same name as that of the class, and it does not have a return type. ~ precedes the identifier to indicate destructor.

Destructor

```cpp
#include <iostream>
using namespace std;

class Test {
public:

    Test() {
        cout << "Constructor Called" << endl;
    }

    ~Test() {
        cout << "Destructor Called" << endl;
    }
};

int main() {
    Test t;

    return 0;
}
```

Advantages of Using Parameterized Constructors

- Parameterized constructors offer various benefits, which make object initialization more flexible, clean, and efficient. Below we have discussed some of the key advantages.
- It allows flexible object initialization with simplified code, improving readability and maintainability.
- It prevents Object Creation with Invalid Data, where it allows valid data only.
- It reduces the need for setter functions and supports constructor overloading.
- It also provides better memeory management (with Dynamic Memory Allocation), enforces Default Values for Missing Arguments, and overall improves code performance.

Constructor Overloading

Constructor overloading or multiple parameterized constructors in C++, is a concept, which allows users to define multiple constructors in the same class with different parameter lists, where each constructor can initialize the object differently depending on the arguments passed while creating an object.

Key points

- **Same Constructor Name** − All overloaded constructors must have the same name as the class.
- **Different Parameters** − Constructors must have a different number or types of parameters.
- **Flexibility** − Constructor overloading also gives flexibility during object initialization by allowing the same class to be instantiated in various ways.

Constructor Overloading

```cpp
#include <iostream>
using namespace std;

class Student {
public:
    int id;
    string name;
    double cgpa;

    Student() {
        cout << "Default Constructor" << endl;
    }

    Student(int id , string name)
    {
        this->id = id;
        this->name = name;
    }

    void display1()
    {
        cout << id << " " << name << endl;
    }
    Student(int id , string name, double cgpa)
    {
        this->id = id;
        this->name = name;
        this->cgpa = cgpa;
    }
    void display2()
    {
        cout << id << " " << name << " " << cgpa << endl;
    }

};
```

```cpp
int main() {
    Student ob1;

    Student ob2(101, "Prime");
    ob2.display1();

    Student ob3(102, "University", 3.97);
    ob3.display2();
    return 0;
}
```

Copy Constructor

A **copy constructor** is a member function *that initializes an object using another object* of the same class. Copy constructor takes a reference to an object of the same class as an argument.

A **copy constructor** is a type of constructor that creates an object using another object of the same class. The process of initializing members of an object through a copy constructor is known as **copy initialization**. It is also called **member-wise initialization** because the copy constructor initializes one object with the existing object, both belonging to the same class on a member-by-member copy basis.

- Initialize one object from another of the same type.
- Copy an object to pass it as an argument to a function.
- Copy an object to return it from a function.

Copy Constructor

A **Copy Constructor** creates a new object, which is an exact copy of the existing object. The compiler provides a default Copy Constructor to all the classes.

Syntax:

class-name (class-name &object_name){}

Definition:

class_name :: class_name(class_name &object_name)

Copy Constructor

```cpp
#include <iostream>
using namespace std;

class Student {
public:
    int val;

    Student(int x) {
        val = x;
    }
    // Copy constructor
    Student(Student& a) {
        val = a.val;
    }
};

int main() {
    Student a1(20);

    // Creating another object from a1
    Student a2(a1);

    cout << a2.val;
    return 0;
}
```

When a custom copy constructor is needed

- **When the class uses dynamic memory:** If your class allocates memory using new, you need a custom copy constructor to perform a deep copy. The default shallow copy will result in both the original and the new object pointing to the same memory location, which can lead to the classic "double free" error when both objects go out of scope.

- **To ensure a truly separate copy:** Even without dynamic memory, a custom copy constructor allows you to define specific logic for how an object's members are copied. This gives you complete control over the initialization process.

- **To prevent modification of the original object:** When a deep copy is performed, the new object can be modified without affecting the original, ensuring the original object's state remains intact.

When a copy constructor is called

- When initializing an object from another object: For example, MyClass obj2 = obj1; or MyClass obj2(obj1);

- When passing an object by value to a function: The copy constructor is invoked to create a copy of the object that is passed as a parameter.

- When a function returns an object by value: A copy is made to return the object from the function's scope.

Characteristics of Constructors

- The name of the constructor is the same as its class name.
- Constructors are mostly declared as public member of the class though they can be declared as private.
- Constructors do not return values, hence they do not have a return type.
- A constructor gets called automatically when we create the object of the class.
- Multiple constructors can be declared in a single class
- In case of multiple constructors, the one with matching function signature will be called.

Inside the class:

```
class Sample
{
public:
Sample()
{cout<< “Message from constructor”; }
~Sample()
{cout<< “Message from destructor”; }
} ;
```

Outside the class:

```
class Sample
{
public:
Sample()
~Sample()
};
Sample :: Sample()
{cout<< “Message from constructor”; }
Sample :: ~Sample()
{cout<< “Message from constructor”; }
```

Default Constructor Example:

```
#include<iostream>
using namespace std;
    class Sample{
    public:
    Sample()
    {cout<< “Message from constructor”; }
    ~Sample()
    {cout<< “Message from destructor”; }
    } ;
    void func1(void){
    cout<<“\n Creating Sample Object in func1”;
    Sample smp; //constructor is calling here
                    //destructor is calling here
    }
Void main(void)
{
cout<< “\n Creating Sample object in main()”;
Sample smp; //constructor is calling here
cout<<“\n Calling func1”;
func1();
cout<<“\n\n func1 is called;
                    // destructor is calling here
}
```

Output:

```
Calling Sample object in main()
Message from constructor
Calling func1
Creating Sample object in func1
Message from Constructor
Message from Destructor

func1 is called
Message from Destructor
```

Parameterized Constructor

Parameterized constructor declaration: **classname(argument list)**

Parameterized constructor object declaration:
class_name object_name = class_name(value of argument_list)
class_name object_name(value of argument list)

```
#include<iostream>
using namespace std;
class Sample{
int x, y;
Public:
Sample(int a, int b)
{ x=a;
  y=b;}
void add(){
cout<< "\n X+Y = "<<x+y<<endl;}
};
```

```
void main(void)
{
Sample smp=Sample(20,30);
smp.add();
}
```

Notes: There is no parameter in destructor